

The main GRUB2 difference between BIOS and UEFI

While most of the configuration syntax and tools remain the same between BIOS and UEFI boots, some of the following small changes do apply.

Linux16/initrd16 vs linuxefi/initrdefi

The configuration commands to load a kernel and initial RAM disk switch from *linux16* and *initrd16* to *linuxefi* and *initrdefi*, respectively.

The change is necessary since kernels need to be loaded differently on a UEFI system than on a BIOS system. The *grub2-mkconfig* command automatically recognises a UEFI system and makes the correct changes.

/boot/grub2 vs /boot/efi/EFI/redhat

On UEFI systems the directory holding all grub2 configuration files and extras is moved to */boot/efi/EFI/redhat*. This is a location on the ESP, making it accessible to the UEFI firmware.

grub2-install

While a *grub2-install* command is more than capable of installing a boot loader onto a UEFI system, it should never be used directly. Calling *grub2-install* on a UEFI system will generate a new */boot/efi/EFI/redhat/grubx64.efi* file. On Red Hat Enterprise Linux 7 systems, this file should be the prebaked version from the *grub2-efi-package*.

The *grub2-install* command will also register a bootable target in the UEFI firmware using this updated *grubx64.efi* application, instead of the *shim.efi* application.

⚠ Warning: Using *grub2-install* will install a custom *grubx64.efi*. If a system was set up for a secure boot, this will cause the boot to fail.

/boot/grub2/grub.cfg vs /boot/efi/EFI/redhat/grub.cfg

Included in the move from */boot/grub2* to */boot/efi/EFI/redhat/* is the move of the main GRUB2 configuration file. The symbolic link */etc/grub.cfg* is also moved to */etc/grub2-efi.cfg*.

Reinstalling GRUB2 on UEFI-based machines

If, for some reason, any of the files needed to boot in */boot/efi* have been removed, or if the partition has been recreated, these files can be recovered by reinstalling both the *grub2-efi* and *shim* packages, as follows:

```
# yum reinstall grub2-efi shim
```

If */boot/efi/EFI/redhat/grub.cfg* has been removed, it can be restored with the following command:

```
# grub2-mkconfig -o /boot/efi/EFI/redhat/grub.cfg.
```

If the Red Hat entry has been removed from the UEFI boot menu, booting from the disk that has an installation of Red Hat Enterprise Linux 7 will automatically add it back.

The UEFI based chain

In order to work with secure boot systems, where signatures on boot loaders, kernels, etc, can be checked by the firmware to authenticate that only unmodified software is run, Red Hat Enterprise Linux 7 uses the shim UEFI bootloader.

The *shim.efi* application is signed with a key trusted by most UEFI firmware. When started, it will attempt to load *grubx64.efi* using the normal UEFI firmware calls. If the UEFI firmware refuses to load the application due to a bad signature, shim will attempt to verify the application using other keys compiled into it, or a user-generated key stored in UEFI NVRAM (machine owner key (MOK)). When attempting to start a signed application for which no keys have been registered, the *MokManager.efi* application is started automatically so that an end user can register a personal key.

The *shim.efi* application also registers a new UEFI system call that can be used by GRUB2 to verify kernel signatures.

If *shim.efi* is not registered with the UEFI firmware, booting from the disk that contains the ESP will launch the */boot/efi/EFI/BOOT/BOOTX64.efi* application. This, in turn, will launch the *fallback.efi* application, which automatically registers the *shim.efi* application and boots the system, based on the settings in */boot/efi/EFI/redhat/BOOT.CSV*.

Early root shell

In some cases, startup tasks can take far longer than anticipated, or even fail to complete after minutes or hours. To help debug these types of problems, the *systemctl list-jobs* command can show all current jobs that *systemd* is executing, allowing an administrator to stop or kill them, or fix the reason these jobs are taking so long.

To obtain a root shell during startup, an administrator can enable the *debug-shell.service*. This service will start a virtual console with a logged-in root shell attached to it on */dev/tty9*, very early during boot—during the startup of the *sysinit.target*.

Using this early root shell, an administrator can analyse, debug and sometimes remedy a failing service or unit.

⚠ Warning: Do not leave the *debug-shell.service* service enabled and running after debugging is done. Anyone with console access, either physical or via a management card or KVM switch, will have full, unrestricted root access to the machine.



By: Kshitij Upadhyay

The author is RHCSA and RHCE certified, and loves to write about new technologies. He can be reached at upadhyayk04@gmail.com.